

GUIs in Java

(Program Design & Implementation 2)

Living Room

Brightness: 70%

- Ceiling Light: 70%
- Window Shades: Open
- Floor Fan: Speed 2

Choose scene...

```
<VBox spacing="10" alignment="TOP_CENTER" ...>
  <Label text="Living Room" styleClass="header" />
  <HBox spacing="10" alignment="CENTER">
    <Label text="Brightness:" />
    <Slider value="70" max="100" min="0" ... />
    <Label text="70%" />
  </HBox>
  <VBox spacing="5" styleClass="panel" ...>
    <Label text="Ceiling Light: 70%" />
    <Label text="Window Shades: Open" />
    <Label text="Floor Fan: Speed 2" />
  </VBox>
  <HBox spacing="10" ...>
    <ComboBox promptText="Choose scene..." />
    <Button text="Activate" styleClass="action-btn" />
  </HBox>
</VBox>
```

Brightness: 70%

- Ceiling Light: 70%
- Window Shades: Open
- Floor Fan: Speed 2

- Choose scene... **Activate**

Activate

CS 3100: PROGRAM DESIGN AND IMPLEMENTATION II

LECTURE 29: GUIs IN JAVA

©2026 Jonathan Bell & Ellen Spertus, CC-BY-SA

LEARNING OBJECTIVES

After this lecture, you will be able to:

1. Explain the difference between sequential and event-driven programming
2. Apply the Model-View-Controller pattern to separate UI from business logic
3. Build a simple JavaFX interface using FXML, controllers, and standard components
4. Use JavaFX properties and binding to keep the View synchronized with the Model
5. Apply accessibility practices (accessible text, keyboard navigation, focus management) in JavaFX

YOUR CODE HAS BEEN TELLING USERS WHAT TO DO – NOW THEY TELL YOU

Sequential (CLI) – you're in control

```
System.out.println("Enter room name:");  
String room = scanner.nextLine(); // blocks here  
System.out.println("Enter brightness (0-100):");  
int brightness = scanner.nextInt(); // blocks here  
light.setBrightness(brightness);  
System.out.println("Set to " + brightness + "%");
```

Your code decides the order. The user waits for you.

Event-driven (GUI) – the user is in control

```
brightnessSlider.setOnMouseReleased(e -> {  
    // runs when user releases the slider  
});  
sceneButton.setOnAction(e -> {  
    // runs when user clicks "Activate Scene"  
});  
// code continues immediately  
// doesn't wait for any interaction
```

The user decides the order. Your code waits for them.

This is called **inversion of control**. You don't call the framework – the framework calls you.

THE EVENT LOOP: YOUR CODE IS A GUEST IN SOMEONE ELSE'S HOUSE

Every GUI framework has an event loop at its core:

```
// Pseudocode - what the framework does
while (applicationIsRunning) {
    Event event = waitForNextEvent();    // block until click, keypress, timer...
    EventHandler handler = findHandler(event); // look up your registered callback
    handler.handle(event);               // call YOUR code
}
```

What this means for you:

- Your code runs *inside* the framework's loop — you are a guest
- If your handler takes too long, the entire UI freezes (no events can be processed)
- In JavaFX, this loop runs on the **JavaFX Application Thread** — all UI updates must happen here

JAVA'S THREE GUI TOOLKITS TELL THE STORY OF UI EVOLUTION

	AWT (1995)	Swing (1997)
Approach	Wraps native OS widgets	Draws its own widgets in Java
Consistency	Looks different on each OS	Identical everywhere
Code	<pre>Button b = new Button("Go");</pre>	<pre>JButton b = new JButton("Go")</pre>
Strength	Native look and feel	Cross-platform consistency
Weakness	Lowest common denominator	Looks dated, no CSS/FXML

We'll use **JavaFX** because its architecture — scene graphs, declarative UI, CSS styling, property binding — reflects patterns used in modern frameworks (React, SwiftUI, Flutter).

POLL: HAVE YOU EVER CREATED A JAVA GUI?

- A. I've used Swing**
- B. I've used JavaFX**
- C. I've used a different Java framework**
- D. I don't know which Java framework I used**
- E. I've modified Java GUIs**
- F. No**
- G. I created all the code myself**
- H. I used Scene Builder**
- I. I used AI**

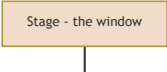


Text **espertus** to 22333 if the URL isn't working for you.

<https://pollev.com/espertus>

Choose all that apply.

THE JAVAFX API: STAGE, SCENE, AND SCENE GRAPH



Stage - the window

Stage is the OS-managed application window — title bar, minimize, maximize, close.

Scene is the content inside the window. A Stage holds one active Scene at a time. You can swap Scenes at runtime — like a browser tab loading a different page.

The **Scene Graph** is a tree of *nodes* — every widget and layout container is a node. You build the tree; the framework walks it to render, handle focus, and dispatch events.

Leaf nodes are widgets (Labels, Buttons, Sliders). **Container nodes** (VBox, HBox) control layout. *Analogy:* FXML is to JavaFX what HTML is to the browser.

YOUR FIRST JAVAFX APPLICATION

```
public class SceneItAllApp extends Application {

    @Override
    public void start(Stage stage) {
        // Build the scene graph – a tree of nodes
        Label title = new Label("Living Room");
        title.setStyle("-fx-font-size: 20px; -fx-font-weight: bold;");

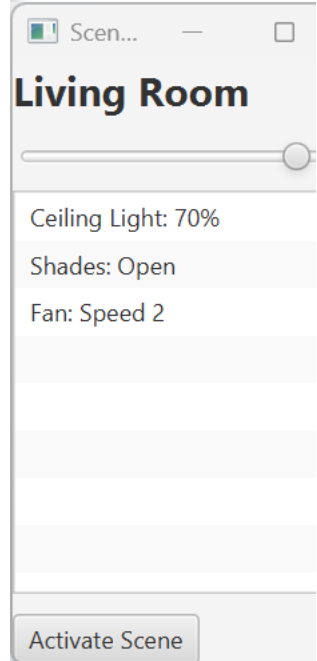
        Slider brightness = new Slider(0, 100, 70);
        Button activate = new Button("Activate Scene");
        activate.setOnAction(e -> System.out.println("Scene activated!"));

        ListView<String> devices = new ListView<>();
        devices.getItems().addAll("Ceiling Light: 70%", "Shades: Open", "Fan: Speed 2");

        VBox root = new VBox(10, title, brightness, devices, activate);

        // Wrap the tree in a Scene, put the Scene in a Stage
        Scene scene = new Scene(root, 400, 300);
        stage.setTitle("SceneItAll");
        stage.setScene(scene);
        stage.show();
    }

    public static void main(String[] args) { launch(args); }
}
```



This works — but all the UI construction, event handling, and data are declared programmatically in one method. That won't scale.

<https://github.com/cs3100-spertus-s26/javafx-examples/blob/slide9/app/src/main/java/org/example/demo/SceneItAllApp.java>

FXML: BUILDING THE SAME TREE DECLARATIVELY

Java (imperative)

```
Label title = new Label("Living Room");
title.setStyle("-fx-font-size: 20px;");

Slider brightness = new Slider(0, 100, 70);

ListView<String> devices = new ListView<>();

Button activate = new Button("Activate");
activate.setOnAction(e -> { ... });

VBox root = new VBox(10,
    title, brightness, devices, activate);
```

Build the tree with constructor calls.
Mix structure with behavior.

FXML (declarative)

```
<VBox spacing="10">
  <Label text="Living Room"
        style="-fx-font-size: 20px;"/>

  <Slider min="0" max="100" value="70"/>

  <ListView fx:id="deviceList"/>

  <Button text="Activate"
        onAction="#handleActivate"/>
</VBox>
```

Declare what exists. Behavior lives elsewhere (Controller)

<https://github.com/cs3100-spertus-s26/javafx-examples/blob/slide10/app/src/main/resources/scenes/scene1.fxml>

Same scene graph tree. Different way to define it. FXML separates **structure** (what exists) from **behavior** (what happens).

POLL: WHICH OF THESE ARE PART OF THE MODEL?

- A. the list of devices
- B. the names of the devices
- C. the brightness of the ceiling light
- D. the slider
- E. the text on the button

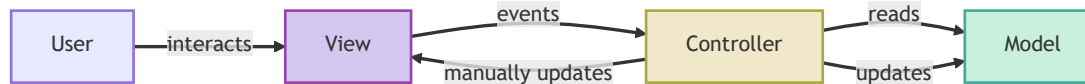


Text **espertus** to 22333 if the URL isn't working for you.

<https://pollev.com/espertus>

MVC IS HEXAGONAL ARCHITECTURE FOR USER INTERFACES

In [L16](#) we learned to separate domain logic from infrastructure so code is testable. MVC applies that principle to GUIs:



The Model is cleanly domain-side — no UI imports, fully testable. But the Controller does *everything else*: it reads from the Model, makes decisions, AND manually pushes updates to View widgets. It straddles the hexagonal boundary. We'll fix this in L30.

Model

Data + business logic. Knows nothing about the UI.

View

What the user sees. Widgets, layout, styling. No business logic.

Controller

Translates user actions into model updates. The thin middleman.

WHAT GOES WRONG WITHOUT MVC

This is what code looks like when UI and business logic are tangled together:

```
// Everything in one place – the "Big Ball of Mud"
dimButton.setOnAction(event -> {
    // Assume there is a single brightness text field that sets
    // the brightness of all lights.
    int brightness = Integer.parseInt(brightnessField.getText());
    // Business logic mixed into the click handler
    if (brightness < 0) brightness = 0;
    if (brightness > 100) brightness = 100;
    for (SmartDevice device : deviceList) {
        if (device instanceof Light) {
            // Network call in the UI handler
            zigbeeController.sendCommand(device.getId(), "brightness", brightness);
        }
    }
    // Logging in the UI handler
    auditLog.record("Brightness set to " + brightness + " by " + currentUser);
});
```

How do you unit test the brightness validation logic? You can't — it's trapped inside a button click handler.

THE MODEL: BUSINESS LOGIC WITH NO UI DEPENDENCIES

```
// Model: Light.java – pure business logic, fully testable
public class Light extends SmartDevice {
    private int brightness; // 0-100
    private boolean isOn;

    public void setBrightness(int level) {
        this.brightness = Math.clamp(level, 0, 100);
        this.isOn = brightness > 0;
    }

    public void toggle() {
        this.isOn = !this.isOn;
    }

    public int getBrightness() { return brightness; }
    public boolean isOn() { return isOn; }
    public String getStatus() {
        return isOn ? getName() + ": " + brightness + "%" : getName() + ": Off";
    }
}
```

```
// You can test this right now – no GUI required
@Test
void brightnessClampedToValidRange() {
    Light light = new Light("Desk Lamp");
    light.setBrightness(150);
    assertEquals(100, light.getBrightness());
}

@Test
void settingBrightnessToZeroTurnsOff() {
    Light light = new Light("Desk Lamp");
    light.setBrightness(0);
    assertFalse(light.isOn());
}
```

THE VIEW: WHAT THE USER SEES (FXML)

JavaFX separates UI structure into FXML — an XML file that declares what widgets exist and how they're arranged:

```
<!-- View: area-dashboard.fxml -->
<?xml version="1.0" encoding="UTF-8"?>
<?import javafx.scene.control.*?>
<?import javafx.scene.layout.*?>

<VBox spacing="10" xmlns:fx="http://javafx.com/fxml"
      fx:controller="sceneitall.AreaDashboardController">

  <Label fx:id="areaNameLabel" style="-fx-font-size: 18px;"/>

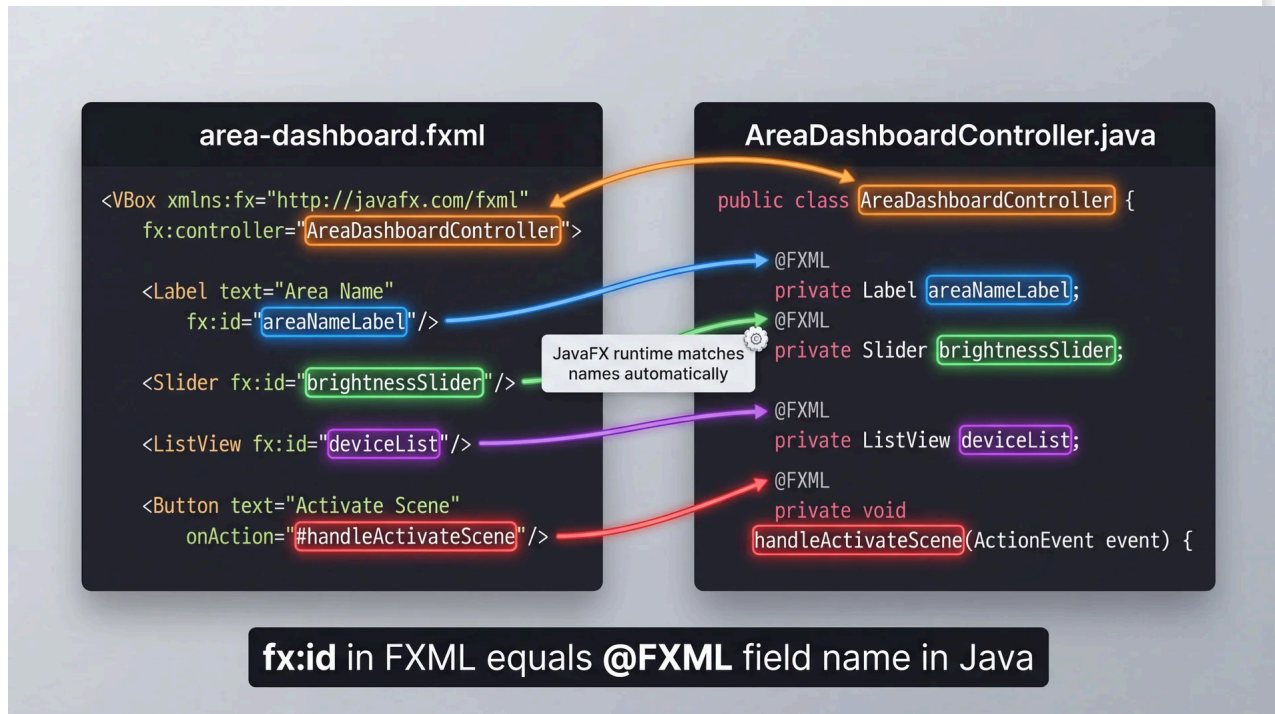
  <HBox spacing="5" alignment="CENTER_LEFT">
    <Label text="Brightness:"/>
    <Slider fx:id="brightnessSlider" min="0" max="100"/>
    <Label fx:id="brightnessValueLabel" text="50%"/>
  </HBox>

  <ListView fx:id="deviceList"/>

  <Button fx:id="sceneButton" text="Activate Scene"
         onAction="#handleActivateScene"/>
</VBox>
```

No business logic here. The View defines **what** appears — not **what it means**.

HOW FXML AND JAVA FIND EACH OTHER: @FXML WIRING (IMAGE)



The framework does the wiring — you just make the names match.

HOW FXML AND JAVA FIND EACH OTHER: @FXML WIRING (CODE)

```
<?xml version="1.0" encoding="UTF-8"?>
<!-- areadashboard.fxml -->

<?import javafx.scene.control.*?>
<?import javafx.scene.layout.*?>

<VBox spacing="10" xmlns:fx="http://javafx.com/fxml"
      fx:controller="sceneitall.AreaDashboardController">

    <Label fx:id="areaNameLabel" style="-fx-font-size: 18px;"/>

    <HBox spacing="5" alignment="CENTER_LEFT">
        <Label text="Brightness:"/>
        <Slider fx:id="brightnessSlider" min="0" max="100"/>
        <Label fx:id="brightnessValueLabel" text="50%"/>
    </HBox>

    <ListView fx:id="deviceList"/>

    <Button fx:id="sceneButton" text="Activate Scene"
           onAction="#handleActivateScene"/>
</VBox>
```

```
public class AreaDashboardController {

    @FXML
    private Label areaNameLabel;

    @FXML
    private Slider brightnessSlider;

    @FXML
    private ListView<String> deviceList;

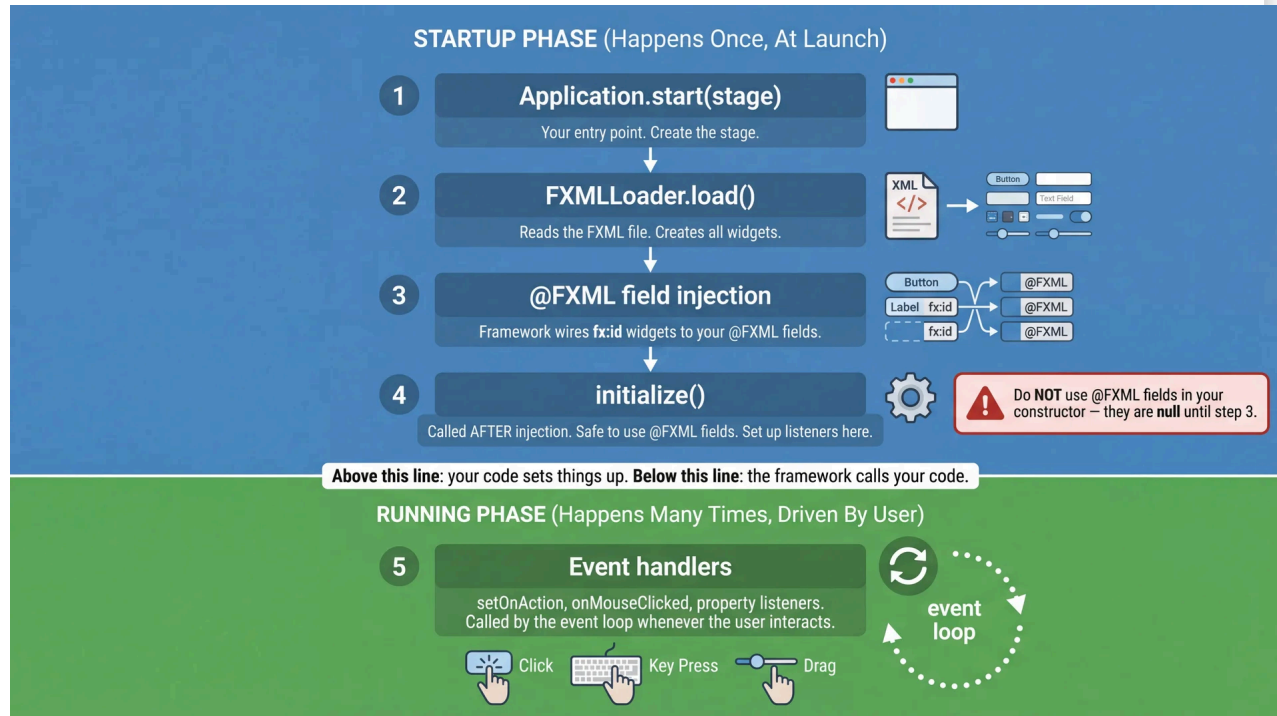
    @FXML
    public void handleActivateScene() {
        System.out.println("Scene activated!");
    }

    public void initialize() {
        deviceList.getItems().addAll(
            "Ceiling Light: 70%",
            "Shades: Open",
            "Fan: Speed 2");
    }
}
```

<https://github.com/cs3100-spertus-s26/javafx-examples/blob/slide17/app/src/main/resources/sceneitall/areadashboard.fxml>

<https://github.com/cs3100-spertus-s26/javafx-examples/blob/slide17/app/src/main/java/sceneitall/AreaDashboardController.java>

THE JAVAFX COMPONENT LIFECYCLE: WHAT RUNS WHEN



THE CONTROLLER: THE TRANSLATOR BETWEEN USER ACTIONS AND BUSINESS LOGIC

```
public class AreaDashboardController {
    @FXML private Label areaNameLabel;
    @FXML private Slider brightnessSlider;
    @FXML private Label brightnessValueLabel;
    @FXML private ListView<String> deviceList;
    private Area model; // the Controller creates and depends on the Model

    @FXML
    private void initialize() {
        Area livingRoom = new Area("Living Room", List.of(new Light("Ceiling Light", 50),
            setModel(livingRoom);

        // Update Label as slider moves
        brightnessSlider.valueProperty().addListener((obs, oldVal, newVal) -> {
            brightnessValueLabel.setText(newVal.intValue() + "%");
            model.setAllLightsBrightness(newVal.intValue()); // delegate to Model
            updateDeviceList();
        });
    }

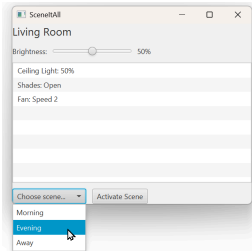
    public void setModel(Area model) {
        this.model = model;
        areaNameLabel.setText(model.getName());
        updateDeviceList();
    }

    @FXML
    private void handleActivateScene() {
        model.activateScene("Evening"); // delegate to Model
        updateDeviceList();
    }

    private void updateDeviceList() {
        deviceList.getItems().setAll( model.getDevices().stream().map(SmartDevice::getStat
    }
}
```

<https://github.com/cs3100-spertus-s26/javafx-examples/blob/slide19/app/src/main/java/scenitall/AreaDashboardController.java>

ADDING SCENE SELECTION



```
<!-- areadashboard.fxml -->

<VBox spacing="10" xmlns:fx="http://javafx.com/fxml"
      fx:controller="sceneitall.AreaDashboardController">

    <Label fx:id="areaNameLabel" style="-fx-font-size: 18px;"/>

    <HBox spacing="5" alignment="CENTER_LEFT">
        <Label text="Brightness:"/>
        <Slider fx:id="brightnessSlider" min="0" max="100" value="50"/>
        <Label fx:id="brightnessValueLabel" text="50%"/>
    </HBox>

    <ListView fx:id="deviceList"/>

    <HBox spacing="10" alignment="CENTER_LEFT">
        <ComboBox fx:id="sceneSelector" promptText="Choose scene..."/>
        <Button fx:id="sceneButton" text="Activate Scene"
              onAction="#handleActivateScene"/>
    </HBox>
</VBox>
```

<https://github.com/cs3100-spertus-s26/javafx-examples/blob/slide20/app/src/main/resources/sceneitall/areadashboard.fxml>

FOLLOWING A CLICK THROUGH ALL THREE LAYERS

User clicks "Activate Scene" for the "Evening" scene. Trace the code:

```
// 1. VIEW (FXML) – button wired to Controller method
<Button text="Activate" onAction="#handleActivateScene"/>

// 2. CONTROLLER – receives the event, delegates to Model
@FXML private void handleActivateScene() {
    String scene = sceneSelector.getValue();    // read from View
    model.activateScene(scene);                // delegate to Model
    updateDeviceList();                        // refresh the View
}

// 3. MODEL – pure business logic, no UI
public void activateScene(String name) {
    Scene scene = scenes.get(name);            // "Evening" → dim to 30%, close shades
    for (SmartDevice device : devices) {
        scene.applyTo(device);                // sets brightness, shade position, fan st
    }
}

// 4. Back in CONTROLLER – View shows updated state
private void updateDeviceList() {
    deviceList.getItems().setAll(              // "Ceiling Light: 30%", "Shades: 80%", "
        model.getDevices().stream().map(SmartDevice::getStatus).toList());
}
```

The Model never imports `javafx`. The View never calls `activateScene()`. Each layer does one job.

<https://github.com/cs3100-spertus-s26/javafx-examples/tree/slide20/app/src/main>

POLL: WHAT IF A HANDLER IS SLOW?

A user clicks 'Activate Scene' and then immediately drags the brightness slider. The `handleActivateScene()` handler takes 3 seconds to run. What happens to the slider drag?

- A. The slider moves smoothly — the framework handles both events in parallel
- B. The slider drag is queued and processed after `handleActivateScene()` finishes
- C. The slider drag is silently dropped — the event is lost
- D. The app throws a `ConcurrentModificationException`



Text **espertus** to 22333 if the URL isn't working for you.

<https://pollev.com/espertus>

POLL: WHERE SHOULD CODE TO TURN OUT ALL LIGHTS LIVE?

You need to add a 'turn off all lights' feature. Which MVC component should contain the logic for iterating through devices and setting brightness to 0?

- A. The View (FXML file)
- B. The Controller
- C. The Model
- D. The CSS stylesheet



Text **espertus** to 22333 if the URL isn't working for you.

<https://pollev.com/espertus>

POLL: CONTROLLER CONSTRUCTION

Given a `DashboardController` where the constructor and `initialize()` both call `areaNameLabel.setText('Living Room')`, what happens when the FXML is loaded?

```
public class DashboardController {  
  
    @FXML  
    private Label areaNameLabel;  
  
    public DashboardController() {  
        areaNameLabel.setText("Living Room");  
    }  
  
    @FXML  
    private void initialize() {  
        areaNameLabel.setText("Living Room");  
    }  
}
```



Text **espertus** to 22333 if
the
URL isn't working for you.

1. The constructor sets the text; `initialize()` overwrites it with the same value
2. The constructor crashes with a `NullPointerException`; `initialize()` would have worked
3. Both fail — you can only set text from FXML, not from Java
4. The constructor works; `initialize()` is never called because the text is already set

POLL: UNIT TESTING

You want to write a unit test that verifies that

- if the user selects the 'Evening' scene
- then the device list will show 'Ceiling light: 30%'

How does this work with our MVC architecture?

A. The Model doesn't have an `activateScene` method — you'd need to add one

B. The Controller's `handleActivateScene()` depends on `@FXML` widgets, so you can't call it without starting the full JavaFX runtime

C. Unit tests can't verify `String` values in lists — you'd need an E2E test

D. There's no problem — you can test the Controller the same way you test any Java class



Text **espertus** to 22333 if the URL isn't working for you.

<https://pollev.com/espertus>

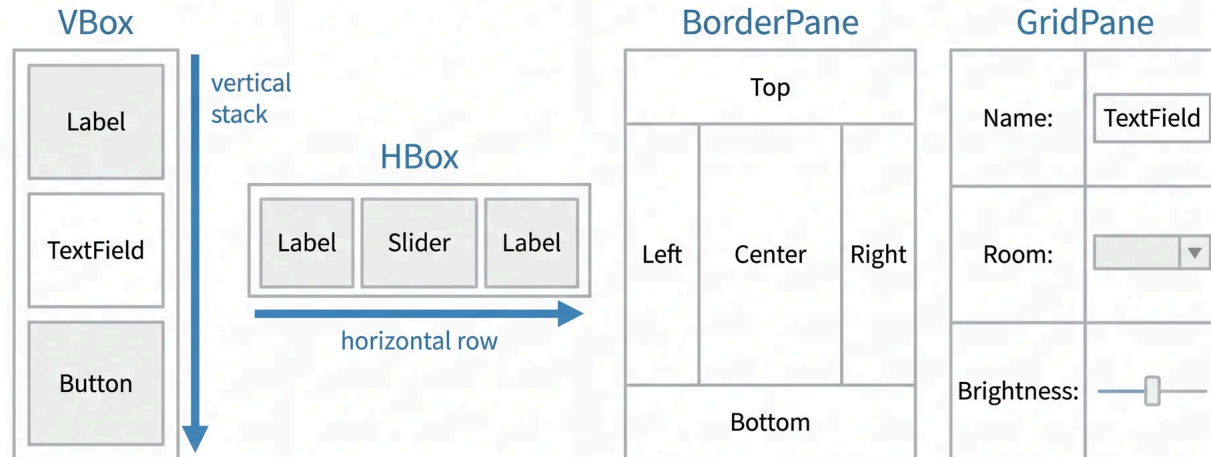
JAVAFX COMPONENTS ARE THE STANDARD COMPONENTS FROM L28

Remember from [L28](#): standard components get accessibility for free. These are the real buttons, not the fakes:

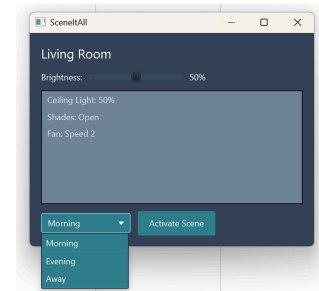
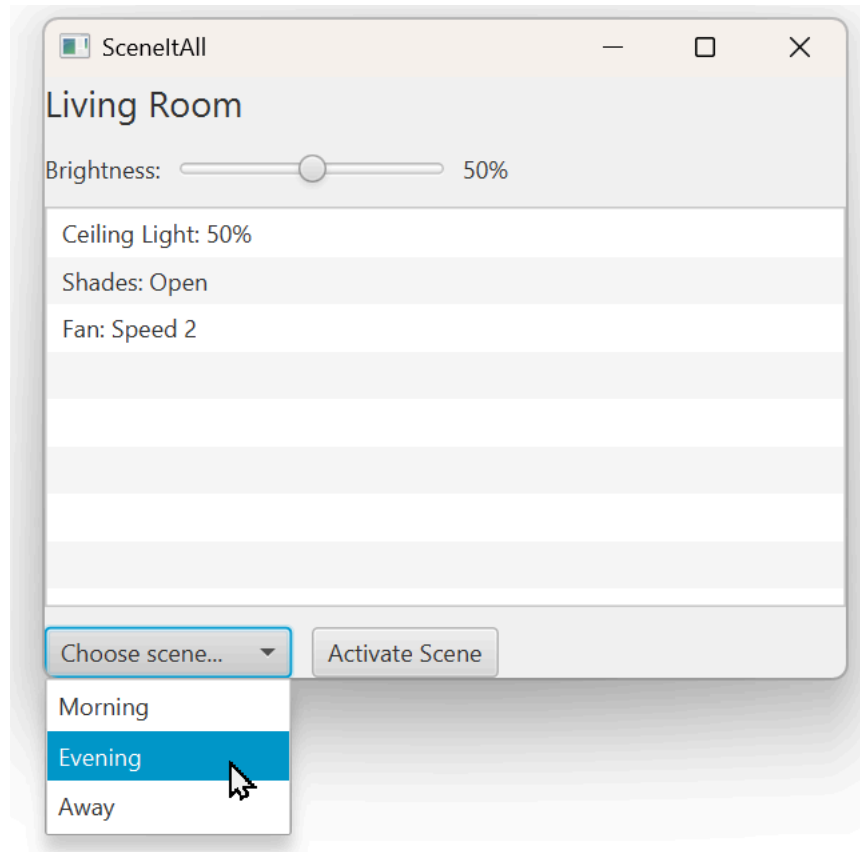
Component	What it does	SceneItAll use case	Accessibility
<code>Button</code>	Clickable action	"Activate Scene"	Screen reader: "Activate Scene, button"
<code>Label</code>	Display-only text	"Living Room"	Screen reader reads the text
<code>Slider</code>	Continuous value	Brightness control	Arrow keys adjust, announced as "Brightness, slider, 75%"
<code>ComboBox</code>	Dropdown selection	Choose a scene	Arrow keys navigate options
<code>ListView</code>	Scrollable list	Device list	Arrow keys navigate, Enter selects
<code>CheckBox</code>	Toggle on/off	Device power	Space to toggle
<code>TextField</code>	Text input	Rename a device	Screen reader: "Device name, text field"
<code>Spinner</code>	Numeric up/down	Fan speed (1-4)	Arrow keys increment/decrement

LAYOUT CONTAINERS: HOW COMPONENTS ARRANGE THEMSELVES

JavaFX Layout Containers



CAN WE MAKE IT LOOK BETTER?



STYLING WITH CSS

```
/* styles.css */
```

```
.root {  
  -fx-font-family: "Inter", sans-serif;  
  -fx-base: #1a1a2e;  
  -fx-accent: #0f3460;  
  -fx-padding: 16px;  
}  
  
.button {  
  -fx-background-color: #0f3460;  
  -fx-text-fill: white;  
  -fx-padding: 8px 16px;  
  -fx-cursor: hand;  
}  
  
.button:hover {  
  -fx-background-color: #16213e;  
}
```

```
.combo-box,  
.combo-box .arrow-button,  
.combo-box .list-cell {  
  -fx-background-color: #0f3460;  
  -fx-text-fill: white;  
}  
  
.combo-box .arrow { -fx-background-color: white; }  
.combo-box:focus {  
  -fx-border-color: #888;  
  -fx-border-width: 1px;  
  -fx-border-radius: 3px;  
}  
  
.list-cell { -fx-text-fill: white; -fx-background-color: transparent; }  
.list-cell:selected { -fx-background-color: #e94560; }  
  
.slider .track { -fx-background-color: #666; }  
.slider .thumb { -fx-background-color: #e94560; }
```

```
<!-- Link the stylesheet in FXML -->  
<VBox stylesheets="@styles.css">
```

<https://github.com/cs3100-spertus-s26/javafx-examples/tree/slide29/app/src/main/resources/sceneitall>

PROPERTIES AND BINDING: WHEN THE MODEL CHANGES, THE VIEW UPDATES AUTOMATICA

JavaFX **properties** are observable values. When they change, anything bound to them updates automatically:

```
// Model with JavaFX properties
public class Light extends SmartDevice {
    private final IntegerProperty brightness = new SimpleIntegerProperty();
    private final BooleanProperty on = new SimpleBooleanProperty();
    private final StringProperty status = new SimpleStringProperty();

    public IntegerProperty brightnessProperty() { return brightness; }
    public BooleanProperty onProperty() { return on; }
    public StringProperty statusProperty() { return status; }

    public void setBrightness(int level) {
        brightness.set(Math.clamp(level, 0, 100));
        on.set(brightness.get() > 0);
        status.set(on.get() ? getName() + ": " + brightness.get() + "%" : getName() + ": 0");
    }
}
```

```
// Controller: bind once, never manually refresh
brightnessSlider.valueProperty().bindBidirectional(light.brightnessProperty());
brightnessValueLabel.textProperty().bind(
    light.brightnessProperty().asString("%d%%")
);
// When light.setBrightness(30) is called anywhere, the slider and label update automatica
```

Bind once. Never call `updateUI()` again. The View and Model stay in sync automatically.

EVERY WIDGET NEEDS A NAME: ACCESSIBLE TEXT IN JAVAFX

From [L28](#): if a screen reader can't announce it, it doesn't exist.
In JavaFX:

```
<!-- Accessible: screen reader says "Living room brightness, slider, 75 percent" -->
<Slider fx:id="brightnessSlider" min="0" max="100"
        accessibleText="Living room brightness"
        accessibleHelp="Use arrow keys to adjust brightness from 0 to 100 percent"/>

<!-- Accessible: screen reader says "Activate scene, button" -->
<Button text="Activate Scene" onAction="#handleActivateScene"/>

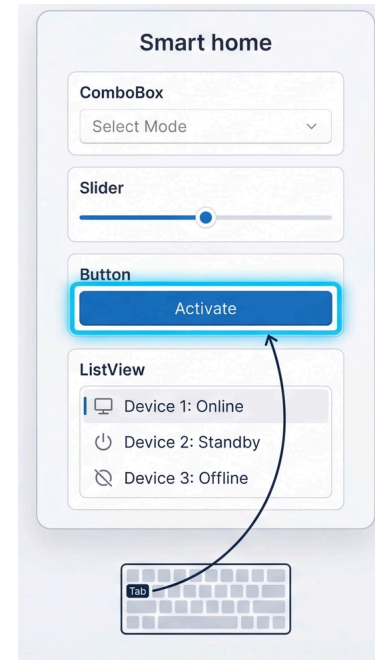
<!-- NOT accessible: screen reader says "button" (no Label!) -->
<Button onAction="#handleToggleLight">
    <graphic><ImageView image="@lightbulb-icon.png"/></graphic>
</Button>

<!-- Fixed: icon button WITH accessible text -->
<Button onAction="#handleToggleLight"
        accessibleText="Toggle desk lamp on or off">
    <graphic><ImageView image="@lightbulb-icon.png"/></graphic>
</Button>
```

KEYBOARD NAVIGATION AND FOCUS MANAGEMENT

Focus is which widget is currently "selected" for keyboard input — the one that will respond when you press Enter, type text, or press arrow keys. Only one widget has focus at a time.

- **Standard components get keyboard navigation for free.** Tab moves focus between widgets in FXML document order. Enter activates buttons. Arrow keys adjust sliders. No extra code.
- **Tab order = FXML document order.** Arrange your FXML so Tab visits elements in a logical sequence.
- **Focus must be visible.** The focused widget shows a highlight ring so keyboard users know where they are. Never remove this with CSS.
- **Dialogs trap focus.** Use JavaFX's built-in `Dialog` and `Alert` — they handle focus trapping and return automatically.



THE PROBLEM MVC DOESN'T SOLVE: TESTABILITY OF THE VIEW

We can unit test the Model — it's pure Java. But the Controller has a problem:

```
// Can we unit test this?
public class AreaDashboardController {
    @FXML private Label areaNameLabel; // needs JavaFX runtime
    @FXML private Slider brightnessSlider; // needs JavaFX runtime
    @FXML private ListView<String> list; // needs JavaFX runtime

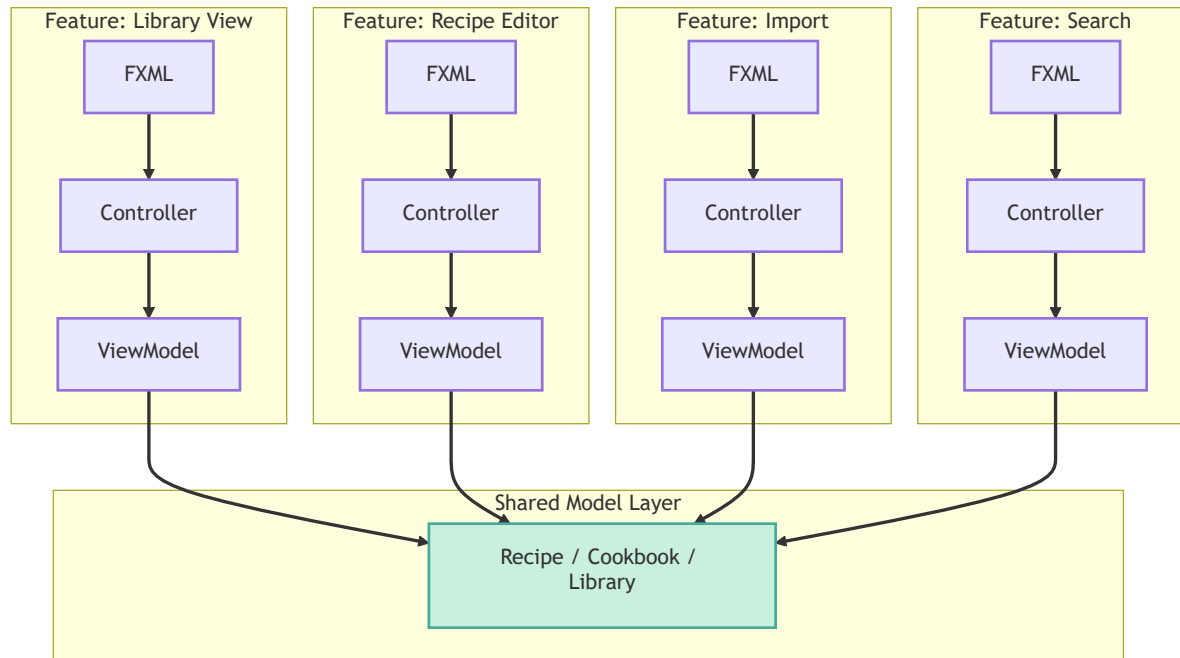
    // To test handleActivateScene(), we'd need to:
    // 1. Start the JavaFX runtime
    // 2. Load the FXML
    // 3. Create all the widgets
    // 4. Simulate a button click
    // That's an integration test, not a unit test.
}
```

The Controller is tangled with JavaFX widgets. We can't test the *logic* without the *framework*.

Could JavaFX solve this by providing a testing library that mocks all your FXML components? Sure — but that's a *lot* of software just to help you test software. There's a simpler answer.

Next lecture (L30): The Model-View-ViewModel (MVVM) pattern solves this by extracting testable logic into a **ViewModel** that has no UI dependencies. No mock framework needed.

YOUR GA1 ARCHITECTURE AT A GLANCE



- Each team member **owns one feature** (View + Controller + ViewModel)
- All features share the **same Model layer** (your domain classes from HW1-HW5)
- **ViewModel interfaces** are provided — they define the contract between features
- Today we used SceneItAll; in GA1 you'll apply the same patterns to **CookYourBooks**

KEY TAKEAWAYS

1. **GUI programming inverts control.** The framework runs the event loop; your code responds to events via callbacks.
2. **MVC separates concerns.** Model = business logic (testable). View = presentation (FXML). Controller = translation (thin).
3. **Use standard components.** Button, Slider, ComboBox, ListView — they get accessibility, keyboard navigation, and screen reader support for free.
4. **FXML separates structure from behavior.** Layout in XML, logic in Java, styling in CSS — three files, three concerns.
5. **Properties and binding keep the View in sync.** Bind once, never manually refresh. The Model and View stay connected automatically.
6. **Accessibility is not extra work — it's using the right components.** Standard components + accessible text + focus management covers the baseline.

LOOKING AHEAD

Lab 12: Hands-on GUI practice

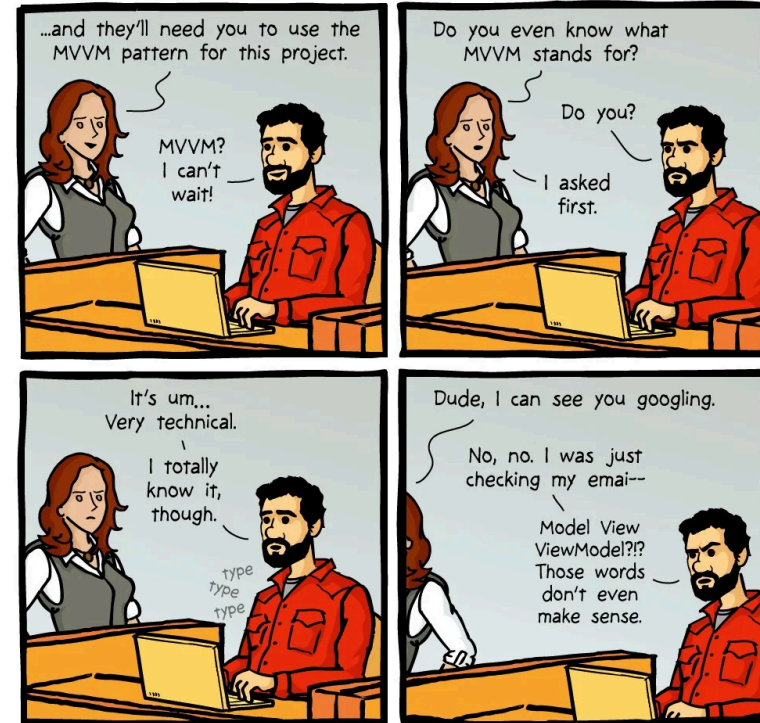
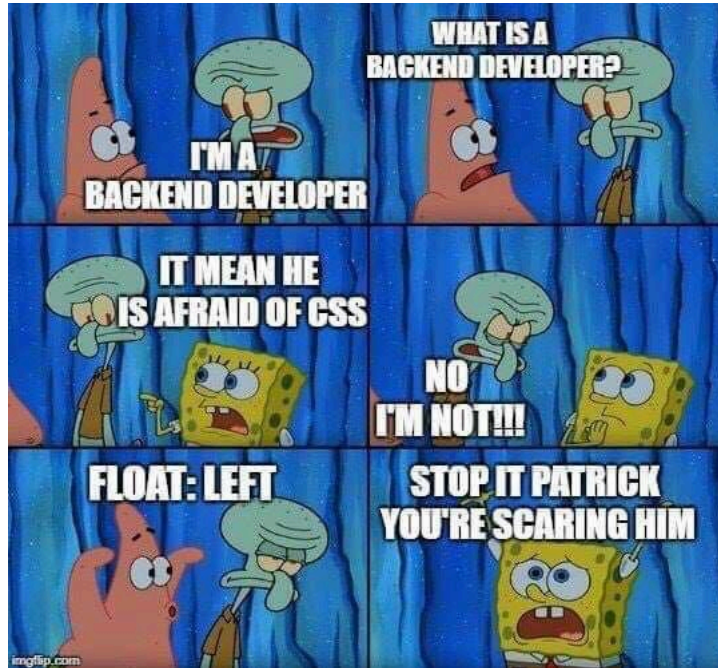
- Build SceneItAll GUI components yourself — using the examples from this lecture and L30
- Bring your laptop with Scene Builder installed

Next up: GUI Patterns and Testing (L30)

- Model-View-ViewModel (MVVM) — the testable evolution of MVC
- Data binding deep dive — observable lists, computed properties
- TestFX — automated end-to-end GUI testing

Today you learned the architecture. Next, you learn to test it. Then you build it for real.

BONUS SLIDE



©2015 Jeff Lofvers

Don't Hit Save - donthitsave.com

